esprit Se former autrement	Année Universitaire : 2025-2026 **Workshops Framework Django 4.1** **UP-Web**

Workshop : Exploration de l'IA : Génération de Texte et d'Images à partir de Texte

Objectif

Ce workshop vise à guider l'utilisation pratique de l'API Gemini et des modèles de machine learning pré-entraînés dans le cadre de l'application Django.

1. Première partie : Génération de la Description d'Événement à partir du Titre.

- Dans cette partie, nous explorerons comment utiliser l'API Gemini et des modèles de machine learning pré-entraînés pour générer des descriptions d'événements à partir de leurs titres. Nous aborderons les étapes suivantes :

1.1. Intégration de l'API Gemini dans l'application Django.

Pour utiliser l'API Gemini dans votre application Django, suivez ces étapes :

- **Obtenez votre clé API Gemini** :

Commencez par naviguer sur https://ai.google.dev/gemini-api/docs pour obtenir votre clé API. Assurez-vous de garder cette clé en sécurité et ne l'incluez pas directement dans votre code source.

- **Installez le SDK Python pour l'API Gemini à l'aide de pip :**

```
pip install -q -U google-generativeai
```

- **Configuration la clé d'API :**

Installer python-dotenv : pip install python-dotenv
Créer un fichier **.env** : Placez vos variables d'environnement dans le fichier **.env** dans le répertoire de l'application principale de votre projet Django.

Charger les variables du fichier .env dans les paramètres de Django : Dans votre fichier de paramètres Django (settings.py), ajoutez le code suivant en haut pour charger les variables :

```
from dotenv import load_dotenv
import os
dotenv_path = os.path.join(os.path.dirname(__file__), '.env')
load_dotenv(dotenv_path)
```

- **Utilisation dans Votre Application Django (views.py) :**

Configurez l'API Key et définissez une fonction pour générer des descriptions basées sur le titre de l'événement :

```
import google.generativeai as genai
import requests

GOOGLE_API_KEY = os.getenv('GOOGLE_API_KEY')

genai.configure(api_key=GOOGLE_API_KEY)
def ai_generate_description(title):
    model = genai.GenerativeModel('gemini-1.5-flash')
    prompt = f"Generate a detailed description for an event with the title in 4
lines: {title}"
    response = model.generate_content(prompt)
    return response.text
```

Ajoutez une vue pour traiter les demandes POST et générer les descriptions d'événements :

```
from django.http import JsonResponse
import json

def generate_description(request):
    if request.method == 'POST':
        data = json.loads(request.body)
        title = data.get('title', '')
        description = ai_generate_description(title)
        return JsonResponse({'description': description})
    return JsonResponse({'error': 'Invalid request'}, status=400)
```

Ajoutez un bouton et un code js dans votre formulaire HTML pour permettre aux utilisateurs de générer des descriptions d'événements en cliquant dessus :

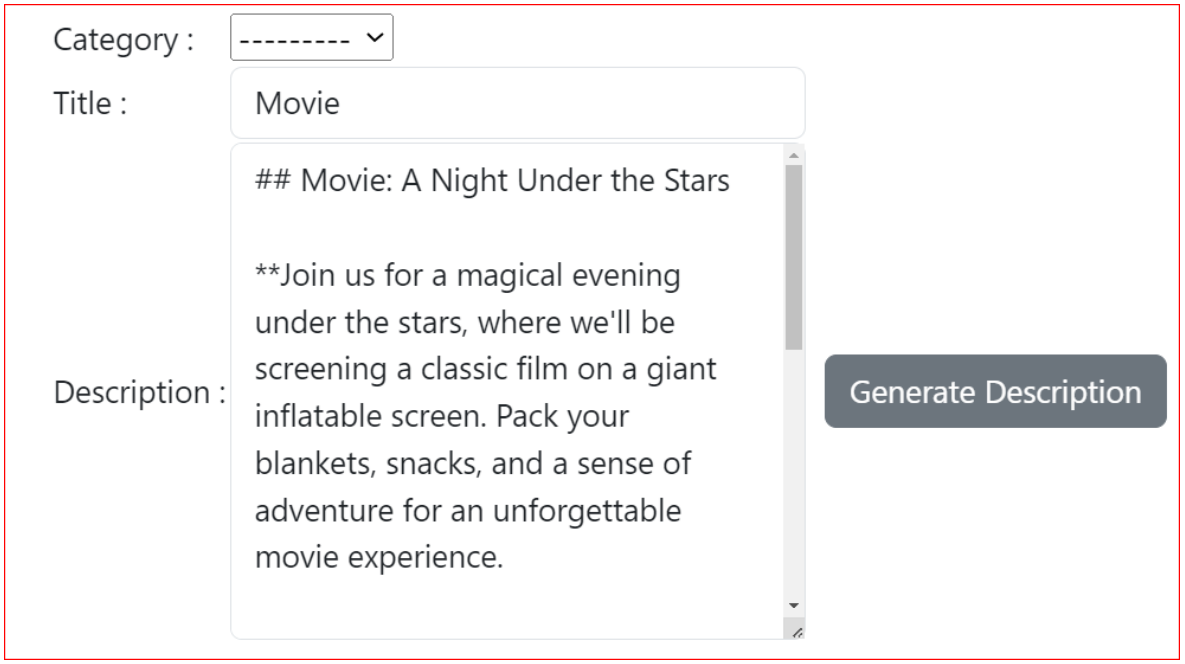
```
 <button type="button" class="mx-2  btn btn-secondary"  id="generate-
description">Generate Description</button>
<script>
    document.getElementById('generate-description').addEventListener('click',
function() {
      const title = document.getElementById('{{ form.title.id_for_label }}').value;
      fetch('{% url "generate_description" %}', {
        method: 'POST',
        headers: {
```

```
          'Content-Type': 'application/json',
          'X-CSRFToken': '{{ csrf_token }}'
        },
        body: JSON.stringify({ title: title })
      })
      .then(response => response.json())
      .then(data => {
        document.getElementById('{{ form.description.id_for_label }}').value =
data.description;
      })
      .catch(error => console.error('Error:', error));
    });
```

Ajoutez une route dans votre fichier **urls.py** pour connecter l'URL à la vue generate_description.

```
    path('generate-description/', generate_description,
name='generate_description'),
```

Ce code configure l'API Gemini avec votre clé API. Il définit une fonction pour générer des descriptions basées sur le titre d'un événement, implémente une vue Django pour traiter les demandes POST et renvoyer les descriptions générées, configure l'URL pour accéder à la vue, et ajoute un bouton HTML avec JavaScript pour générer la description d'un événement à partir de son titre.



2. Deuxième partie : Génération d'Image de l'Événement à partir du Titre

Intégration du Modèle Stable Diffusion de Hugging Face dans l'application Django

Pour utiliser le modèle Stable Diffusion de Hugging Face pour générer une image à partir du titre d'un événement dans votre application Django, suivez ces étapes :

- **Créez un Compte Hugging Face** :

Si vous n'avez pas encore de compte, inscrivez-vous sur Hugging Face.

- **Obtenez votre Token d'Accès :**

Accédez à votre profil sur Hugging Face et obtenez votre token d'accès API. Assurez-vous de garder ce token en sécurité. (.env)

- **Définissez les Variables d'API et les En-têtes :(views.py)**

```
IMAGEGEN_KEY = os.getenv('IMAGEGEN_KEY')
API_URL = "https://api-inference.huggingface.co/models/stabilityai/stable-diffusion-xl-base-1.0"
headers = {"Authorization": f"Bearer {IMAGEGEN_KEY} "}
```

Ajoutez la fonction suivante pour envoyer une requête POST à l'API Hugging Face avec le payload et retourner le contenu de la réponse :

```
def imageGen(payload):
    response = requests.post(API_URL, headers=headers, json=payload)
    return response.content
```

Ajoutez une vue pour traiter les demandes POST et générer l'image d'événement :

```
def generate_image(request):
    if request.method == 'POST':
        # Generate the image
        data = json.loads(request.body)
        title = data.get('title', '')
        image_bytes = imageGen({
            "inputs": f"event for {title}",
        })
        # Save the image to a temporary in-memory file
        image = Image.open(io.BytesIO(image_bytes))
        with io.BytesIO() as output:
            image.save(output, format="JPEG")
            image_data = output.getvalue()

        # Save the image temporarily to the filesystem (e.g., in the media directory)
        image_name = 'generated_image.jpg'
        image_path = 'media/images/' + image_name
        with open(image_path, 'wb') as f:
            f.write(image_data)
        return JsonResponse({'image_url': '/' + image_path})
    return JsonResponse({'error': 'Invalid request method'}, status=400)
```

Modifiez la fonction d'ajout d'un évènement comme suit :

```
def addEvent(req):
    form = EventForm()
    if req.method == 'POST':
        form = EventForm(req.POST,req.FILES)
        if form.is_valid():
            event = form.save(commit=False)
            file_name = os.path.basename('generated_image.jpg')

            print(file_name)
            file_path = os.path.join('media/images', file_name)
            with open(file_path, 'rb') as f:
                event.image.save(file_name, File(f), save=True)
            event.save()
            print (event)

        form.save()
        return redirect('listEvent')
```

```
    return render (req , "event/add.html" , {'form':form} )
```

Ajoutez un bouton et le code js dans votre formulaire HTML pour permettre aux utilisateurs de générer des images d'événements en cliquant dessus :

```
<div id="generated-image-container"></div>
<input type="hidden" name="generated_image_path" id="generated_image_path">
<script>

    document.getElementById('generate-image-button').addEventListener('click',
function() {
      const title = document.getElementById('{{ form.title.id_for_label }}').value;
```

```
            fetch('{% url "generate_image" %}', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'X-CSRFToken': '{{ csrf_token }}'
                },
                body: JSON.stringify(
                  { title: title})
            })
            .then(response => response.json())
            .then(data => {
                if (data.image_url) {
                    document.getElementById('generated-image-container').innerHTML =
'<img  width="100px" src="' + data.image_url + '" alt="Generated Image">';
                    document.getElementById('generated_image_path').value =
data.image_url;
```

```
                    console.log(data.image_url)
                } else {
                    alert('Error generating image');
                }
            })
```

```
            .catch(error => {
                alert('Error: ' + error.message);
            });
        });


</script>
```

Ajoutez une route dans votre fichier urls.py pour connecter l'URL à la vue generate_image.

```
    path('generate-image/', generate_image, name='generate_image'),
```

Ces étapes vous permettent de configurer l'API de Hugging Face pour générer des images à partir de titres d'événements en utilisant le modèle Stable Diffusion. Assurez-vous que votre clé d'accès est correctement configurée et que les bibliothèques nécessaires sont installées.

